

Trabalho Prático.

1 Definição do Projeto

Nesta fase da disciplina, como trabalho prático, deverá ser modificado o projeto implementado na unidade anterior de modo a:

- Considerar a gramática modificada (agora apresentada precedência entre os operadores) disponível na página <https://valdigleis.site>.
- Não necessitar mais de um pré-processador, ou seja, o analisador léxico deve conseguir reconhecer os comentários e os tokens válidos, e assim conseguir gerar a lista de tokens.
- No caso de encontrar erro léxico, o analisador deve informar o erro e a localização exata (linha e coluna) do token com erro.
- O analisador sintático deve ser modificado para conseguir verificar a corretude sintática do código-fonte, detectar erros sintáticos e informar ao usuário, povoar a tabela de símbolos e gerar como saída a árvore sintática abstrata.

2 Tarefas Novas ao Projeto.

Nesta fase, será requerido que o compilador, além do código-fonte, também receba via linha de comando, flags para tornar o uso do mesmo mais amigável e mais útil para depuração das fases de compilação. As seguintes flags de tarefas são consideradas essenciais neste trabalho.

- Flag para indicar ao compilador para imprimir a lista de tokens.
- Flag para indicar ao compilador que o processamento deve parar quando encontrar o primeiro erro léxico, caso contrário, ele deve processar toda a lista de tokens. Se a flag for utilizada, então imprimir o primeiro erro encontrado, senão for utilizada, deve processar toda a lista e imprimir todos os erros léxicos.
- Flag para indicar ao compilador para imprimir a Árvore sintática abstrata.
- Flag para indicar sugestões de correção léxica e sintática.
- Realizar a implementação do analisador sintático de forma que, durante a análise sintática, seja atualizada a tabela de símbolos de forma a recuperar as informações sobre o tipo do símbolo e seu escopo.

- Flag para indicar ao compilador para printar a tabela de símbolos após a fase de análise sintática.
- Implementar a verificação semântica, ou seja, uma série de testes de coerência do funcionamento das construções sintáticas reconhecidas e representadas na AST. Para auxiliar na Seção 4, encontra-se uma descrição de regras semânticas da linguagem.

3 Sobre o trabalho.

Para a realização deste trabalho, deve-se:

1. Manter o grupo da unidade anterior.
2. Criar códigos de teste baseados na nova gramática, que devem ser entregues ao professor como parte da avaliação (2 códigos corretos e 2 com algum tipo de erro).

Como será a correção? A correção será realizada da seguinte forma: o professor irá passar aos grupos (apenas no dia da correção, entre os dias 24 e 28 de junho) 5 códigos, alguns serão corretos e alguns apresentarão algum (ou alguns) tipos de erro. O processamento adequado de cada código irá valer 2,0 pontos.

4 Definições semânticas.

4.1 Semântica dos tipos

A linguagem possui três tipos primitivos: *int*, *boolean* e *int[]*. Além desses, a linguagem permite criar tipos definidos pelo usuário: *class NomeClasse*. Sobre os operadores tem-se que:

1. `-`, `+`, `*` Recebem dois *int* e devem retornar sempre *int*.
2. `<` recebe dois *int* e deve retornar sempre um *boolean*.
3. `&&` recebe dois *boolean* e retorna *boolean*.
4. `!` recebe um *boolean* e retorna *boolean*.
5. `[ ]` indexa um vetor de *int*.
6. `length` retorna um *int*.
7. A atribuição exige compatibilidade de tipos.

4.2 Semântica das classes e da herança

Cada classe pode conter atributos (as variáveis) e(ou) métodos, uma classe vazia, isso é, sem atributos ou métodos, deve ser interpretada como erro semântico. Uma instância criada por *new NomeClasse()*.

As classes podem herdar campos e métodos e os métodos podem ser reescritos por despacho dinâmico, ou seja, se temos:

```
class A {  
    public int f() {  
        return 1;  
    }  
}  
  
class B extends A {  
    public int f() {  
        return 2;  
    }  
}
```

E temos também:

```
A x;  
x = new B();  
x.f();
```

o resultado de $x.f(x)$ deve ser 2.

4.3 Semântica das variáveis

A resolução de nomes de variáveis deve seguir a ordem estabelecida abaixo:

1. Parâmetros do método.
2. Atributos da classe.
3. Atributos herdados.

4.4 Semântica dos comandos

A semântica dos comandos é a mesma que os alunos já conhecem e tiveram contato ao usar linguagens de programação como java, C++ e etc, assim sendo, aqui só será descrito em maiores detalhes os comandos referentes à atribuição.

O comando de atribuição pode ser uma atribuição simples, ou seja, algo da forma $x = E$, neste caso, a avaliação semântica se dá nos seguintes passos:

1. Avalia E .
2. Verifica se os tipos de E e x são compatíveis, então se possível armazena o resultado da avaliação de E em x .

Para o caso da atribuição em um espaço de um vetor, ou seja, uma atribuição da forma $a[i] = E$, a avaliação semântica se dá nos seguintes passos:

1. Avalia a , isso serve para determinar o tamanho de a .
2. Avalia i para determinar se $i <$ que o tamanho de a .
3. Avalia E ;
4. Armazena o resultado E na posição i .

4.5 Semântica das Expressões

A avaliação segue na direção da esquerda para a direita, mas seguindo a precedência dos operadores, sendo tal precedência determinada pela profundidade do operador na AST, quanto mais fundo na árvore, maior sua precedência. Ou seja, $1 + 2 * 3$ deverá ser avaliado como 7 e não como 21.

4.6 Semântica da criação de objetos

A criação de objetos, ou seja, o uso do comando $new Id()$ para algum identificador de classe Id , deve seguir a seguinte semântica de ações.

1. Aloca a memória necessária, ou seja, realiza o cálculo para determinar quanto de memória será necessária e depois solicita a alocação de tal memória.
2. Obrigatoriamente inicializa campos (se houverem) do objeto, ou seja, define valores padrão, por exemplo, poderia ser algo como:
 - Campos do tipo `int` são inicializados com valor 0.
 - Campos do tipo `boolean` são inicializados com valor **false**.
 - Campos que são referências a outros objetos são inicializados, por exemplo, com valor **null**.

4.6.1 Semântica da chamada de métodos e do objeto corrente

A semântica para a chamada de métodos, ou seja, expressões da forma $obj.f(x, y)$ segue da seguinte forma

1. Avalia obj .
2. Avalia os argumentos, se houverem.
3. Cria ambiente local.
4. Associa parâmetros aos argumentos.
5. Executa o corpo do método.

6. Retorna o valor do return.

A referência a um objeto correte, isso é o uso da expressão *this*, algo como por exemplo o código a seguir:

```
<> Java  
  
class A {  
    int x;  
  
    public int get() {  
        return this.x;  
    }  
}
```

A semântica para tal se comporta de forma padrão, ou seja, refere-se ao objeto que fez a chamada do *this*.